CST2120
Web Applications and Databases

Coursework 2 – Social Networking Website

Autumn/Winter term
2024/2025

Date of Submission: 13 December 2024

Student Name: Annah Ndhlovu

Student ID Number: M00942167

Lab Tutor: Tejasvee Devi Boojhawon

Table of Contents

Contents

Table of Contents 2
Table of Figures 2
Introduction 3
The website 3
The webservice 3
Registration 3
Login 5
Conclusion 14
Appropriate Use of AI 14

Table of Figures

Figure 1Initial page 3
Figure 2 Registration 4
Figure 3 http://localhost:8080/M00942167/users 4
Figure 4 login 5
Figure 5 http://localhost:8080/M00942167/login 5
Figure 6 Profile Completion Form 6
Figure 7 http://localhost:8080/M00942167/complete-profile 6
Figure 8 First page 7
Figure 9 Current User Profile 7
Figure 10 http://localhost:8080/M00942167/user-profile/testUser 8
Figure 11 Upload Image Post 8
Figure 12 Current User's Posts 9
Figure 13 User Profiles 10
Figure 14 Display User Profiles 10
Figure 15 Real time search for user Daisy 11
Figure 16 http://localhost:8080/M00942167/search-users?q=Daisy 11
Figure 17 Viewing Daisy's profile 12
Figure 18 http://localhost:8080/M00942167/user-profile/{username} 12
Figure 19 Follow a user 13
Figure 20 Infinite scroll of followed user's posts 13
Figure 21 http://localhost:8080/M00942167/contents/{username}?page=1&limit=5 14

Introduction

This project introduces a photo sharing website, Pixplore created with inspiration from the infamous social networking website, Instagram. I have included in this report brief explanations of the code implemented to get this website functioning. Screenshots of postman and involved webpages have been provided throughout the report to support this code and show its successful functioning.

The website

I pursued this website as it is broad as images have become a fundamental way of communication. This can range from advertising brands in a visually appeasing manner or just expressing oneself by displaying their aesthetics using various pictures. Furthermore, just like its inspiration, Instagram is one of the best platforms of networking due to its photo sharing functionality.

User roadmap

The user begins by filling in a registration form and logging in to have access to the website. Once successful, the user can complete the profile form with their details and then post images for them and the other registered users who follow them to see. The user can also visit the explore section to discover other users to follow and see their posts or search for a specific user they would like to follow.

The webservice

Registration

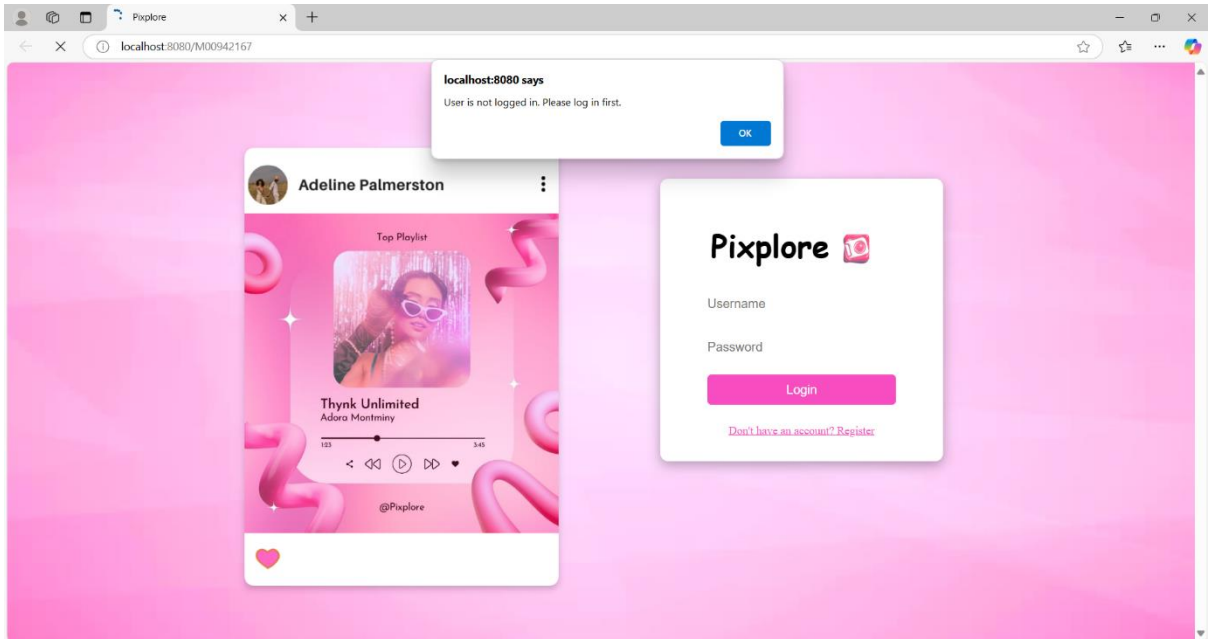

Figure 1Initial page

Upon initial loading of the website using local host, the user is denied further access to the website without logging in, thus the login prompt as shown in figure 1. If the user is a first-time client, they can click on the link redirecting to the registration page.

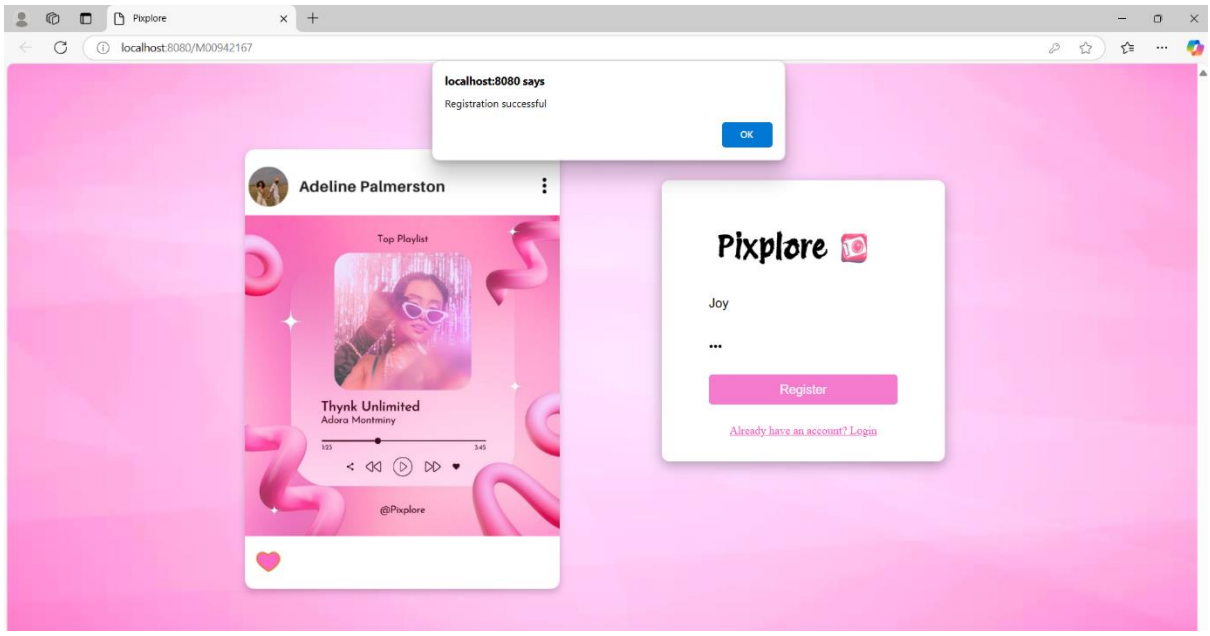

Figure 2 Registration

The user registration feature allows new users to create an account by providing a unique username and password. This functionality ensures that users have their credentials securely stored in the database, enabling authentication for future sessions.

The registration process is implemented on both the client-side and server-side as follows:

- **Client-Side:** The client-side registration functionality allows users to input their desired username and password in the #registerUsername and #registerPassword fields. Upon submission, an AJAX POST request sends this data in JSON format to the server endpoint http://localhost:8080/M00942167/users. If the registration is successful, the user is notified via an alert, and the form transitions to the login interface using the showLoginForm() function. In case of an error, the server's response message is displayed to the user to indicate the issue.
- **Server-Side**: On the server-side, the registration data is received and validated to ensure it meets the required criteria. The server checks the database to confirm the username's uniqueness. If the data is valid, the password is securely hashed, and the user information is stored in the database. Finally, the server responds with a success message if the registration is completed or an error message if any issues arise.

This functionality can further be observed using postman as shown in figure 3 below.

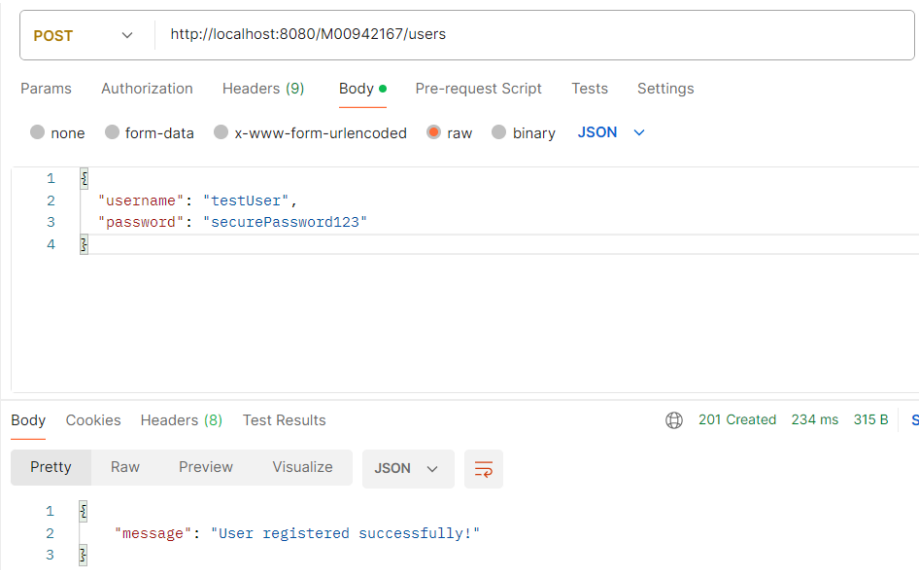

Figure 3 http://localhost:8080/M00942167/users

Login

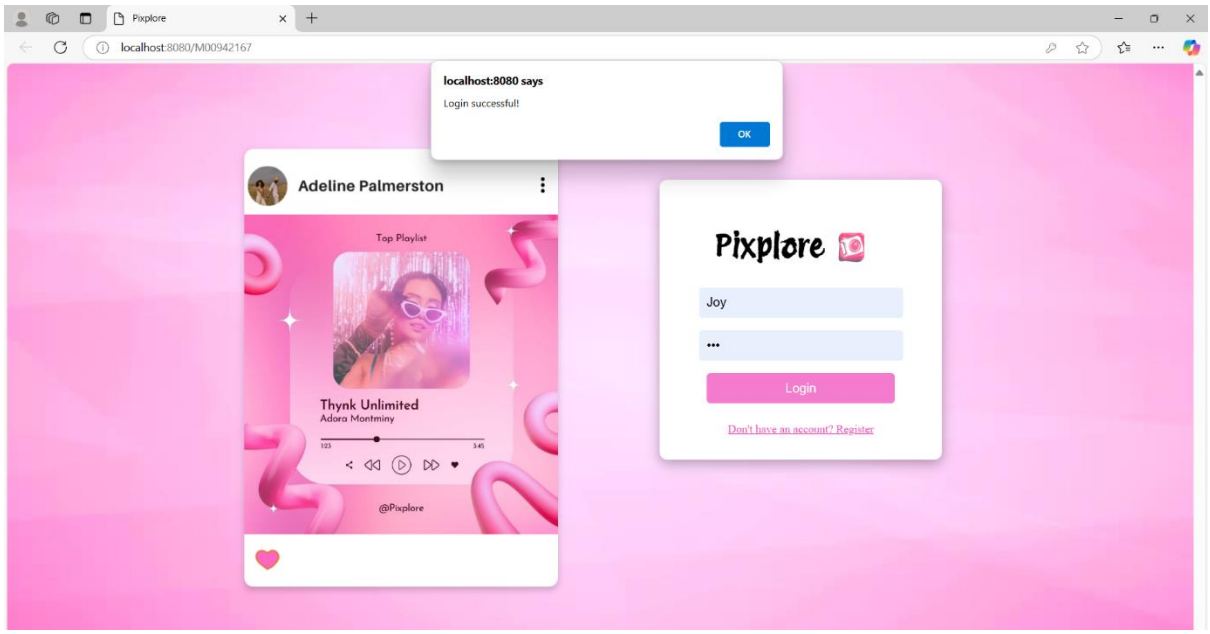

Figure 4 login

The login function handles user login by retrieving the username and password from form inputs and sending them as a JSON object in an AJAX POST request to the http://localhost:8080/M00942167/login endpoint. On success, if a token is returned, it stores the token and username in sessionStorage, notifies the user, and updates the UI to show the profile completion form. If the login fails or the response is unexpected, it displays an error message with details from the server's response.

This functionality is shown further using postman in figure 5 below.

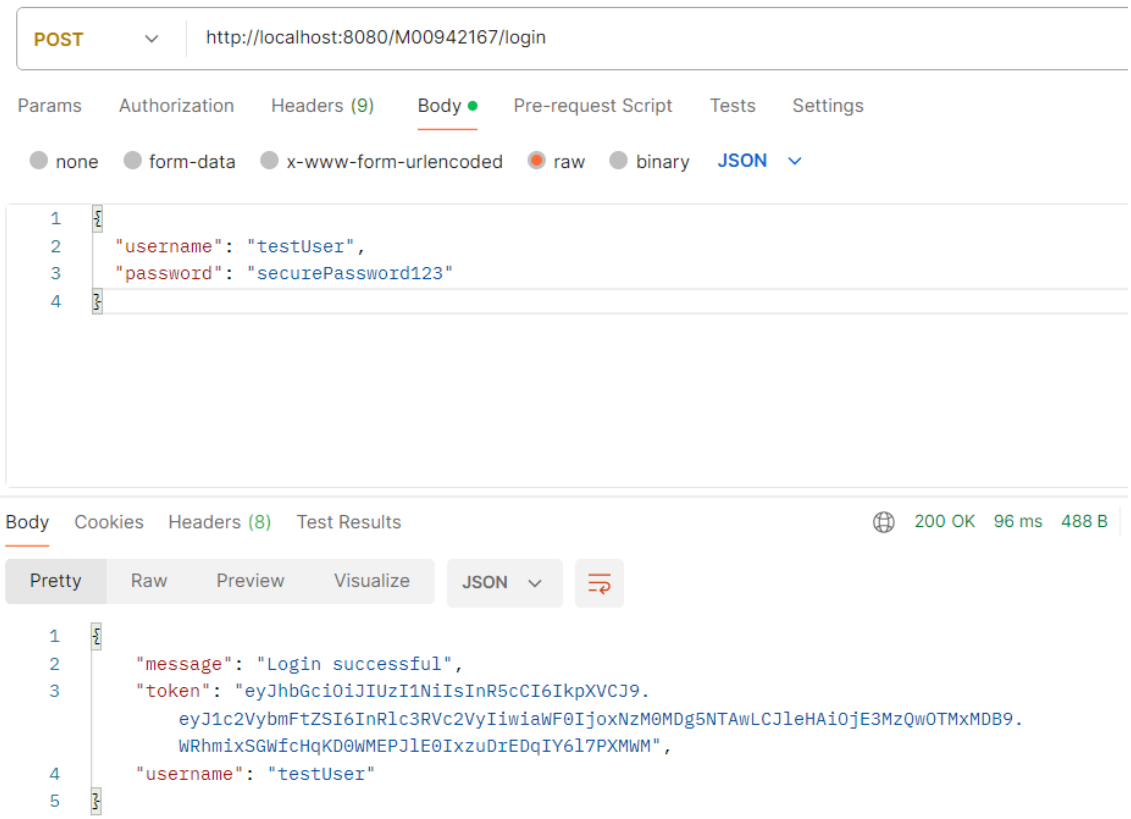

Figure 5 http://localhost:8080/M00942167/login

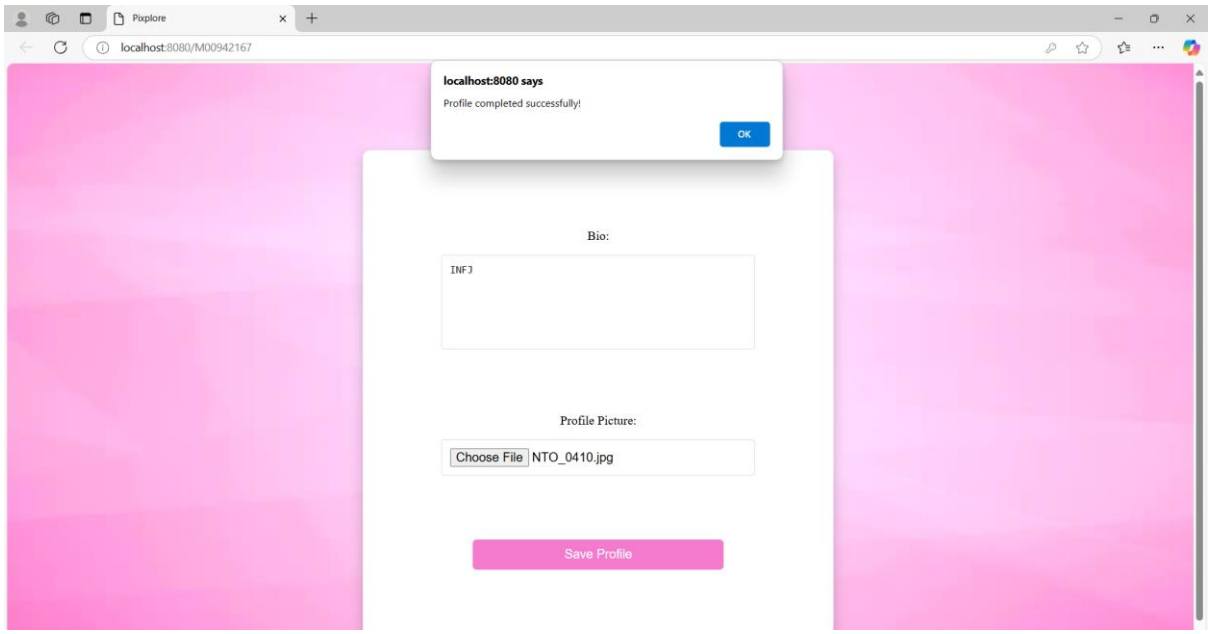

Figure 6 Profile Completion Form

The completeProfile function handles the submission of a user's bio and profile picture to complete their profile. It retrieves the logged-in user's username from sessionStorage, the bio from a text input field, and the profile picture from a file input. If any of these fields are missing, an alert informs the user of the specific missing information. The bio and profile picture are then packaged into a FormData object, which allows sending files along with other data. An AJAX POST request is sent to the endpoint http://localhost:8080/M00942167/complete-profile with the FormData. On success, the profile is marked as complete in sessionStorage, and the user is redirected to the image feed. If there's an error, the user is alerted with details from the server response.

This functionality is further shown using postman in figure 7 below.

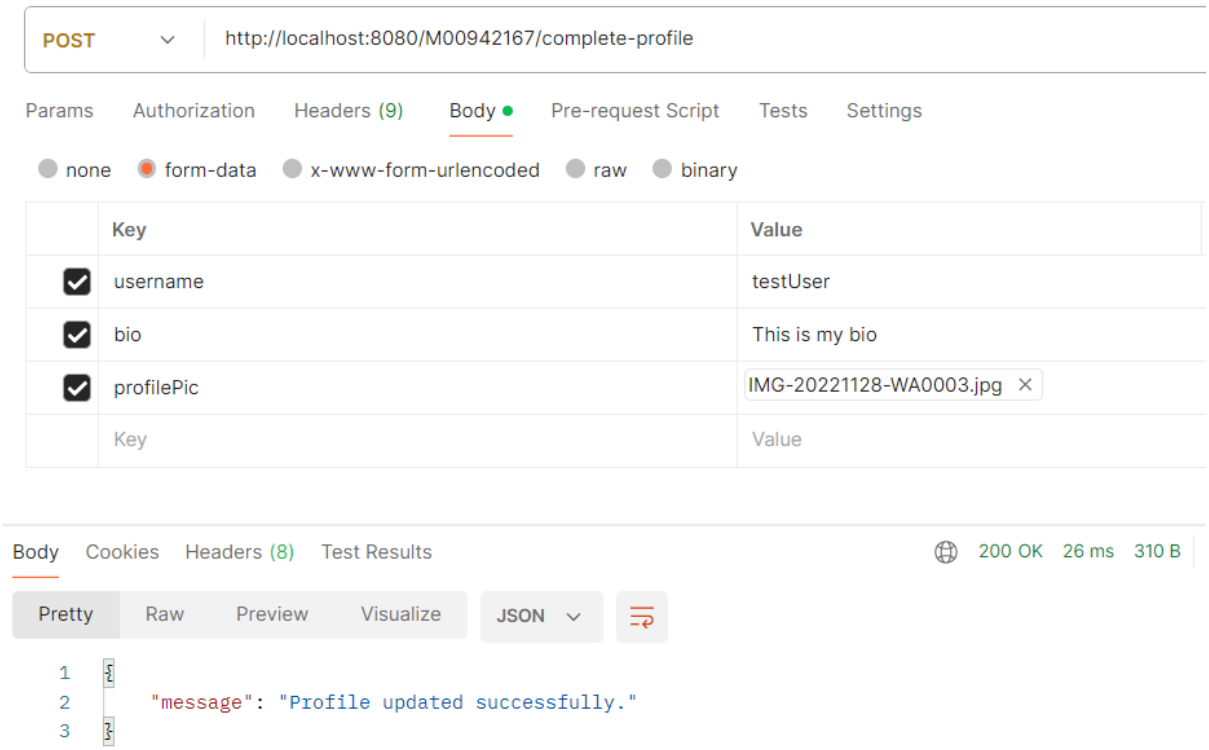

Figure 7 http://localhost:8080/M00942167/complete-profile

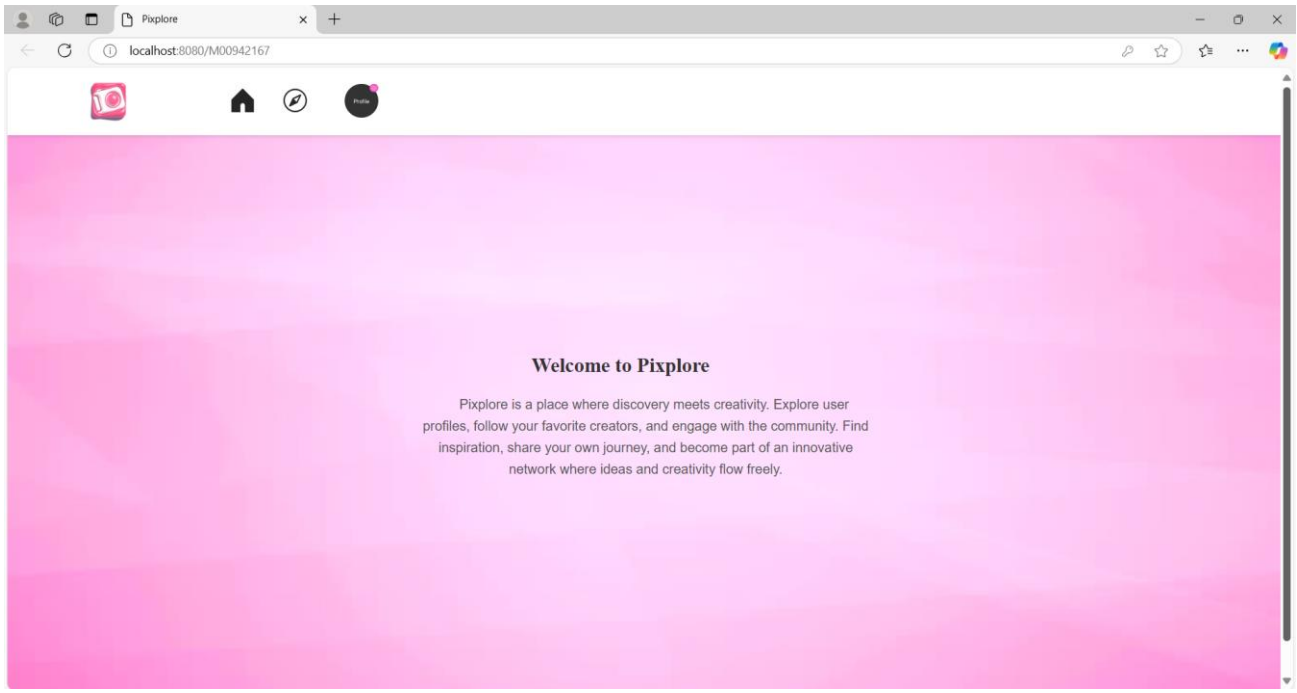

Figure 8 First page

Figure 8 above, shows the initial page in the main website that the user is redirected to following successful profile completion. Displayed is a brief paragraph on the about of the website Pixplore.

The user can then navigate to view their profile details by clicking on the right most icon on the header. Here a function loadCurrentUserProfile retrieves and displays the current user's profile information from the server. It first checks if the user is logged in by checking the sessionStorage for the username. If the user is logged in, it makes a GET request to fetch the user's profile details (including their profile picture and bio) from the server, using the stored token for authentication. Upon success, the profile details are displayed, including an option to create a new post with an image and caption. If the user is not logged in or an error occurs during the request, an appropriate alert is shown. This functionality is shown in figure 9 and 10 below.

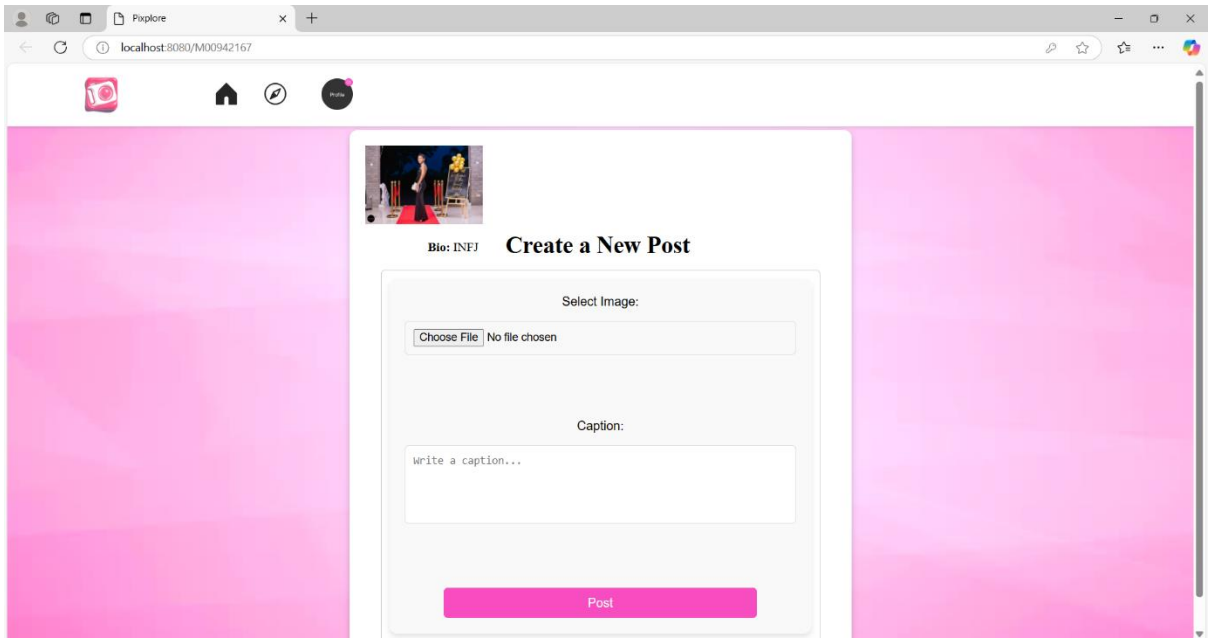

Figure 9 Current User Profile

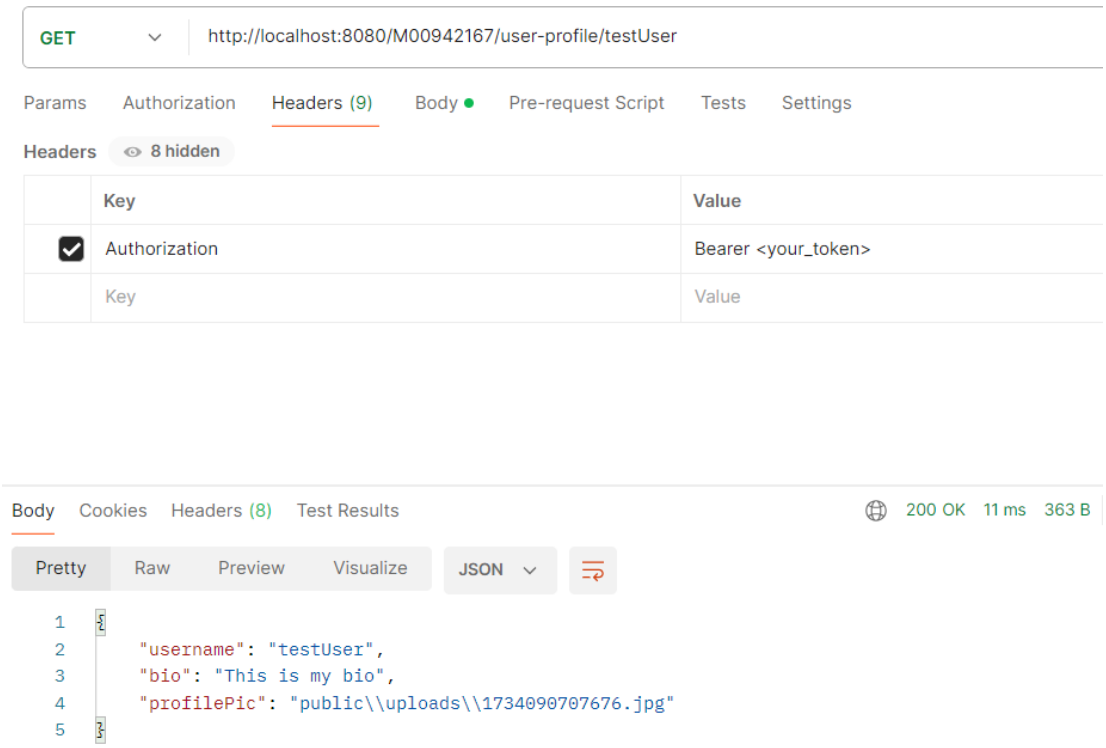

Figure 10 http://localhost:8080/M00942167/user-profile/testUser

On the current user profile, the user can fill in a form to upload a post using the function uploadPost, which handles uploading an image post with a caption. It first checks if the user is logged in by verifying the presence of a JWT token in sessionStorage. If the user is not logged in or if the image or caption is missing, appropriate alerts are shown. If all conditions are met, the function creates a FormData object containing the selected image and caption, and then makes a POST request to the server to upload the content, attaching the JWT token for authentication. Upon successful post creation, a success message is shown to the user, and the form is reset. Additionally, the code includes logic for infinite scrolling, where posts are fetched in pages of 3 posts at a time, starting from page 1. It retrieves the current username from sessionStorage and ensures the user is logged in before proceeding with fetching posts, preventing multiple simultaneous requests by using a loading flag. This is shown below in figure 12..

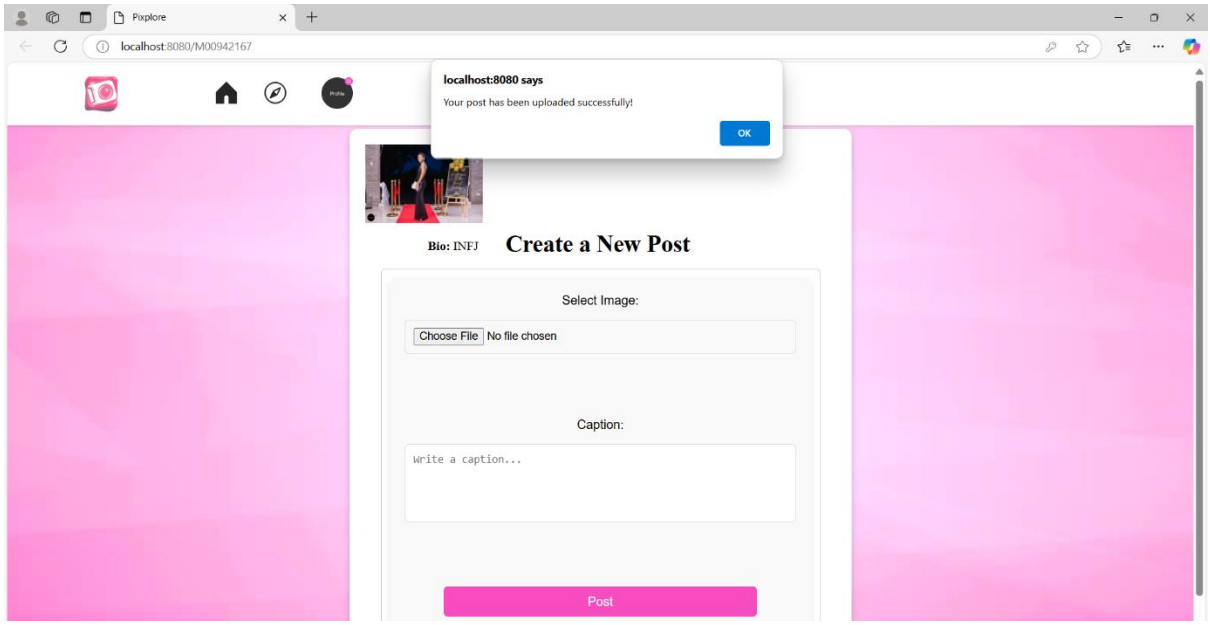

Figure 11 Upload Image Post

After successful uploading of an image post, the user can navigate to the home page via the home icon on the header, this action will invoke a function which handles the fetching and display of user posts with infinite scrolling. It first checks if the user is logged in by verifying the presence of the username, and ensures that multiple fetch requests are not triggered simultaneously by using the isLoading flag. The function makes an HTTP request to the server to fetch a list of posts for the current user, passing the username, page number, and a limit on the number of posts per request. Upon receiving the response, it processes the content by dynamically creating HTML elements for each post, including the username, caption, and image. These elements are then appended to the posts container on the page. Additionally, the function implements infinite scrolling, where new posts are loaded automatically as the user scrolls to the bottom of the page. The currentPage is incremented, and the next set of posts is fetched, ensuring a seamless user experience. If no more posts are available, a message is displayed to inform the user. This implementation is shown in figure 12 below.

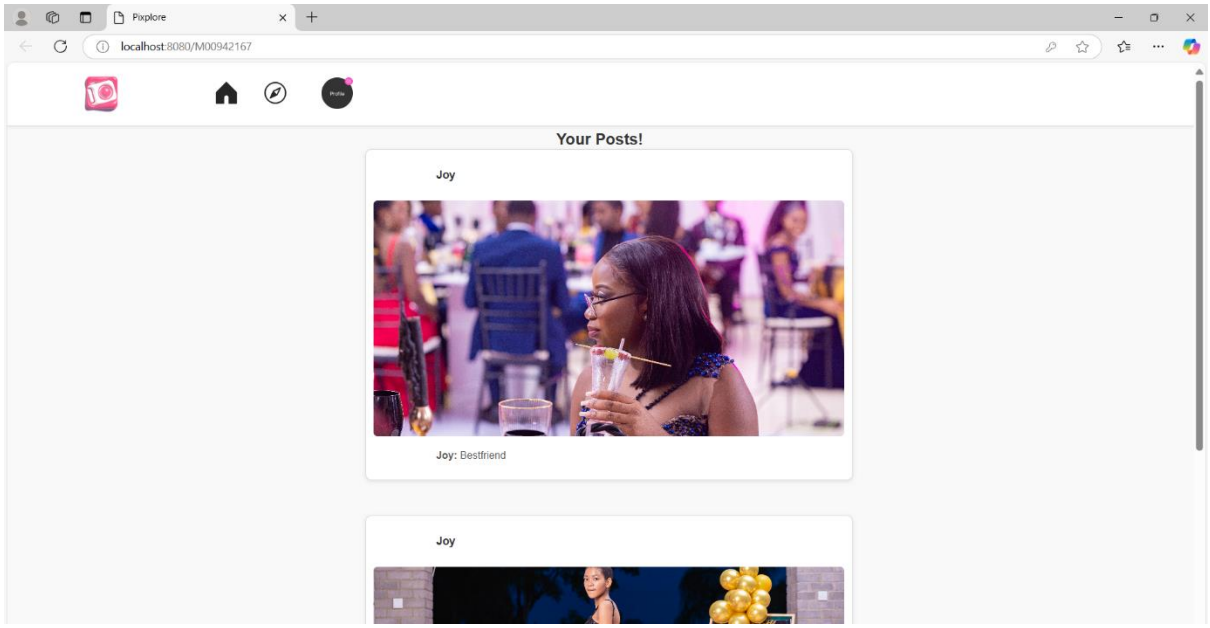

Figure 12 Current User's Posts

The last navigation link is that which leads to the explore section. Here a code sends an AJAX GET request to fetch a list of all users from the server, using an authorization token for authentication. Once the data is received, it dynamically creates and appends user profile cards to the userCardsContainer element, displaying each user's profile picture, username, and bio. If an error occurs during the request, an alert is shown with the error message. See this in figures 13 and 14 below.

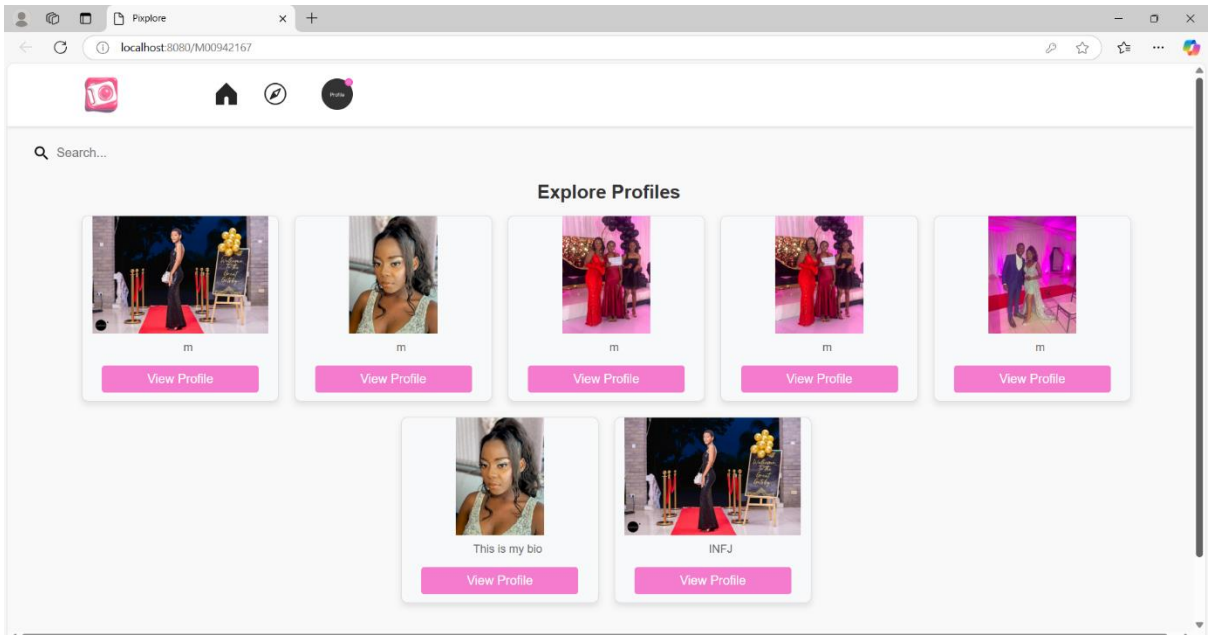

Figure 13 User Profiles

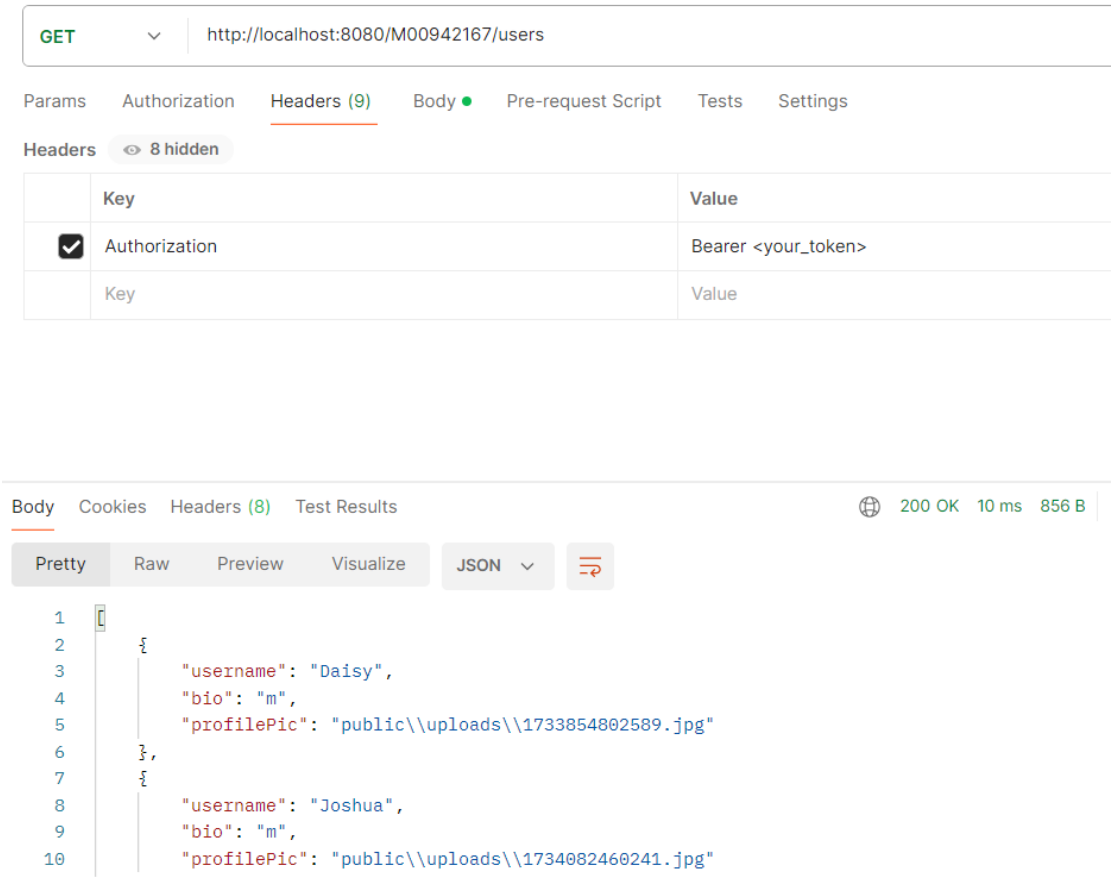

Figure 14 Display User Profiles

When the user profiles are displayed, the user can search for a specific user in real-time as they type into a search box. When the user inputs a search query, an AJAX request is triggered, sending the query to the server's search endpoint (/M00942167/search-users) with the query as a parameter. The request includes a JWT token in the headers for authentication. If the server returns a list of matching users, the code dynamically generates user cards displaying each user's profile picture, username, and bio, and appends them to the #userCardsContainer element. If no users are found, a message indicating "No users found" is displayed. If the search box is cleared, the results are removed from the container. If an error occurs during the AJAX request, an error message is shown to the user. Successful implementation is shown in figures 15 and 16 below.

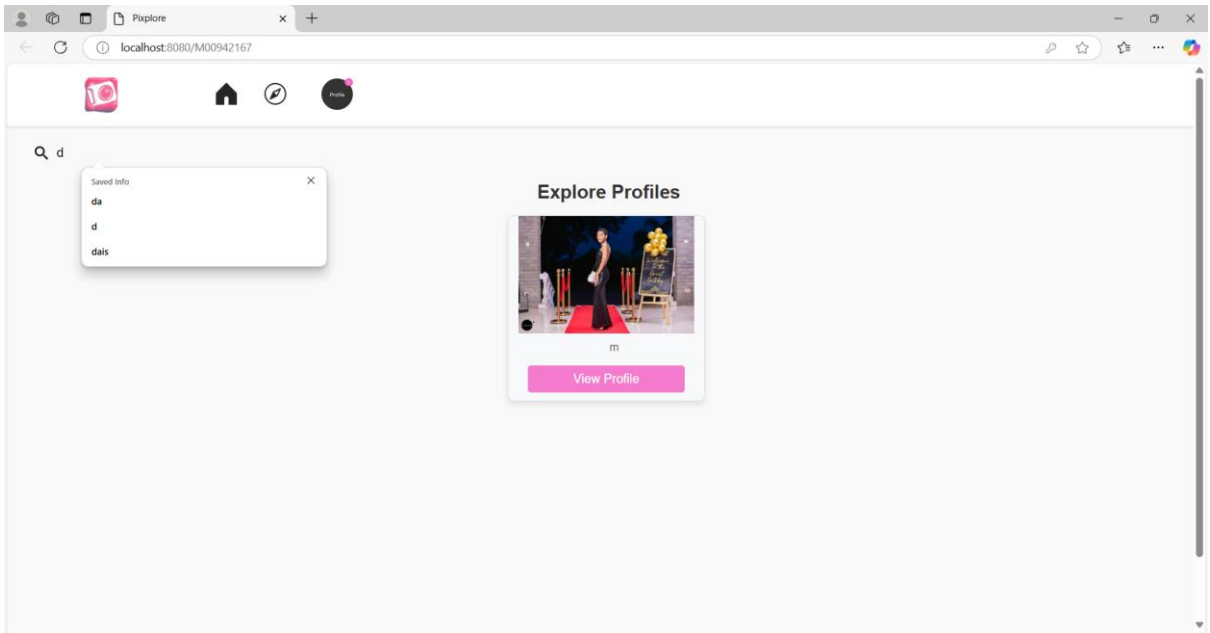

Figure 15 Real time search for user Daisy

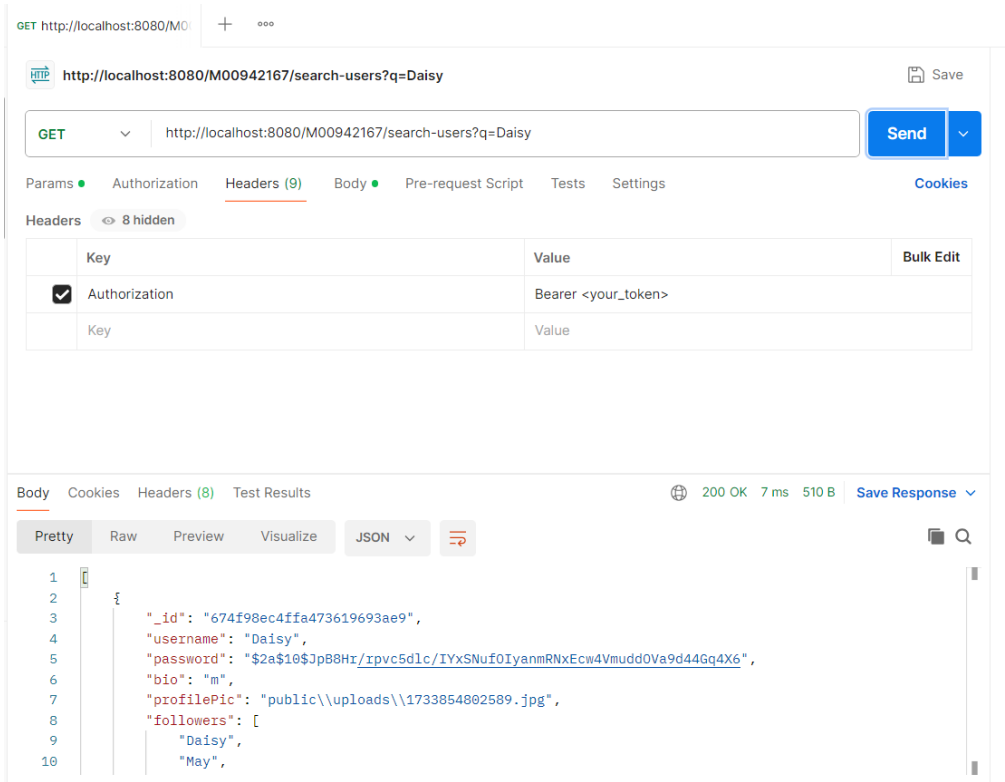

Figure 16 http://localhost:8080/M00942167/search-users?q=Daisy

After the user filters out the user whose profile they searched for, they can click on the view profile button which in this case fetches Daisy's profile data from the server using an AJAX GET request to the endpoint http://localhost:8080/M00942167/user-profile/{username}, where it includes the user's token for authentication. Upon success, it displays Daisy's profile, including her profile picture and bio.

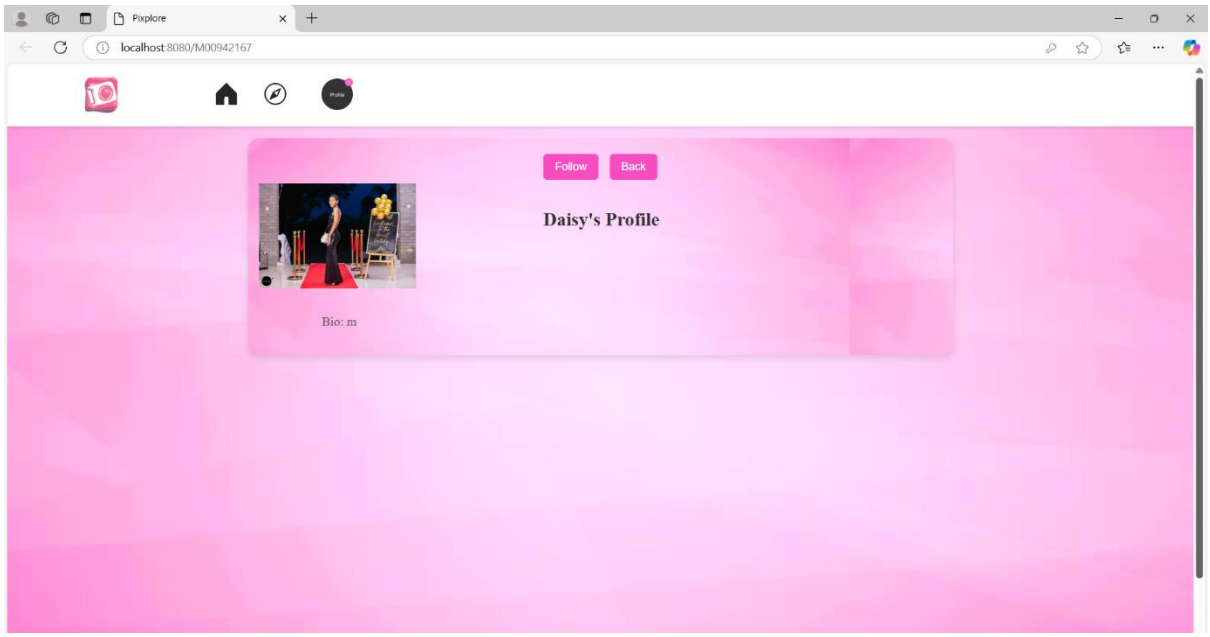

Figure 17 Viewing Daisy's profile

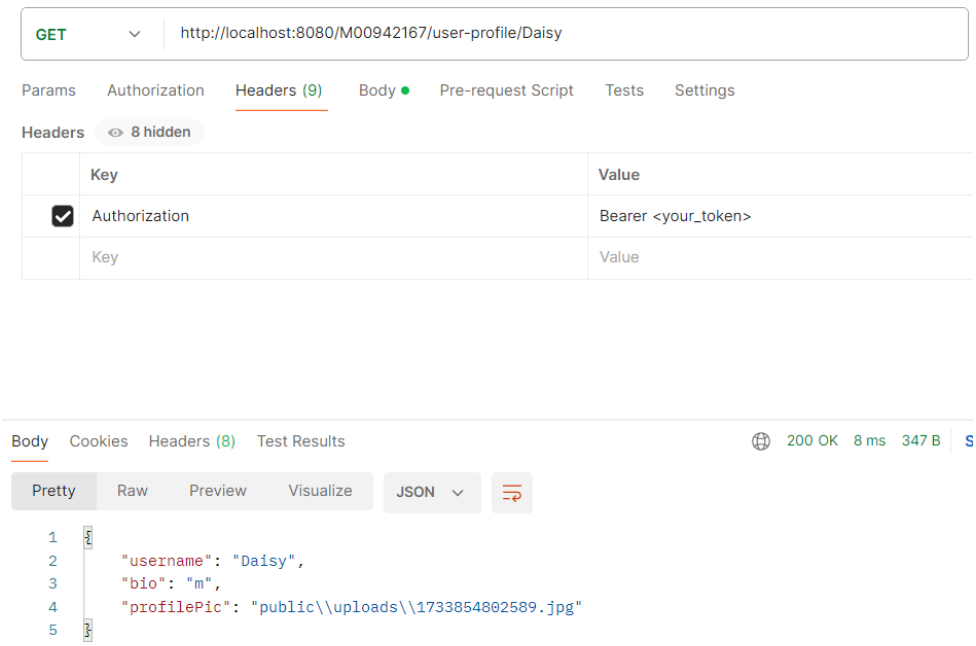

Figure 18 http://localhost:8080/M00942167/user-profile/{username}

The user then has the option to close the user's profile using the back button or to follow the user. If the user decides to follow the user a function extracts the username of the user whose profile is being viewed and checks if the current user is logged in by accessing sessionStorage. If the user is logged in, it sends a POST request to the server with the username and the current user's information, using the Authorization header for authentication. Upon successful response, it updates the follow button text to "Follow" or "Unfollow" based on the user's follow status and, if the user is now following, it fetches and displays their posts. In case of an error, an alert with the error message is shown to the user. This is shown in figure 19 below.

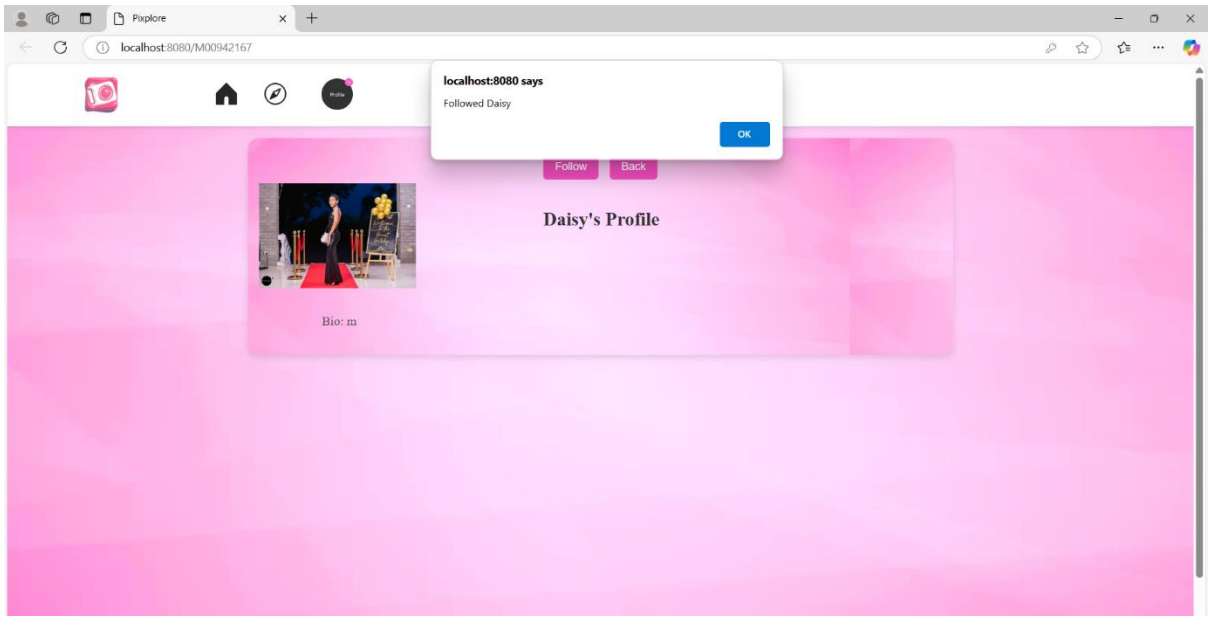

Figure 19 Follow a user

If the current user successfully starts following the target user, the function clears any existing posts displayed on the profile and fetches the target user's posts from the server using the fetchUserPosts function. This function makes a paginated GET request to retrieve the user's posts, which are then displayed in the profile section. The posts are loaded incrementally, and new posts are fetched as the user scrolls down, creating an infinite scroll experience. See this in figures 20 and 21 below.

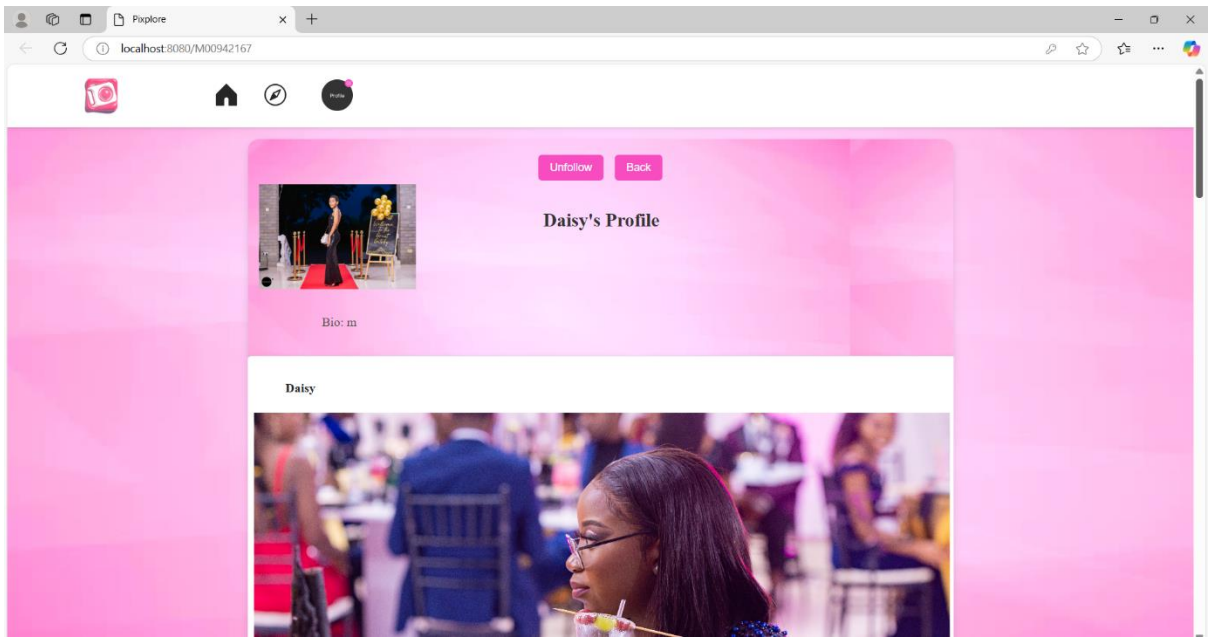

Figure 20 Infinite scroll of followed user's posts

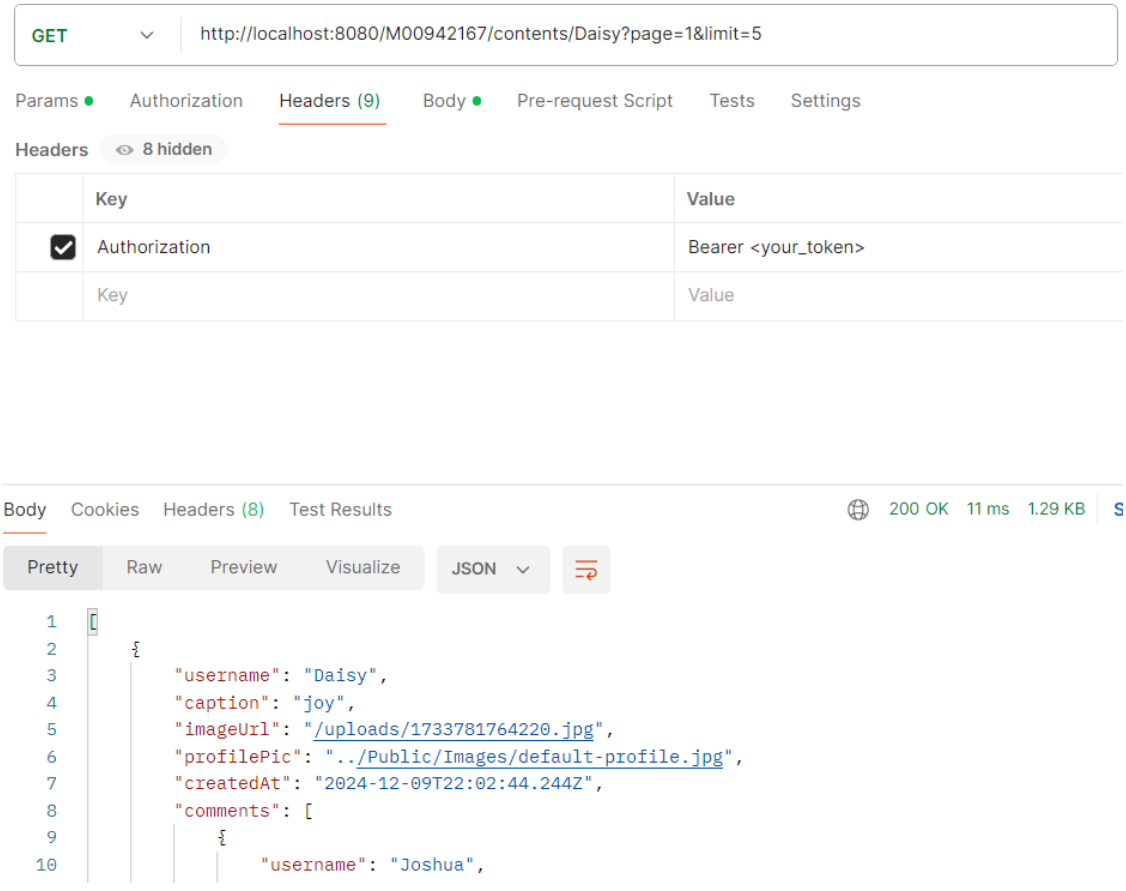

Figure 21 http://localhost:8080/M00942167/contents/{username}?page=1&limit=5

Conclusion

The web browser used to test this game was Microsoft Edge. The code was implemented with the use of JavaScript, CSS and HTML to enable dynamic interactions between users within a social networking platform by integrating features such as user search, profile viewing, following/unfollowing users, and fetching paginated content. The front-end utilizes AJAX requests to communicate with the server, ensuring real-time updates without page reloads. By leveraging session-based authentication with JWT tokens, the system ensures secure and personalized interactions, while pagination ensures efficient loading of large sets of data. This approach not only enhances user engagement but also provides a smooth, interactive experience.

Appropriate Use of AI

Use of AI was limited to understanding complex code functionalities, understanding unfamiliar terms, and mainly debugging where errors arose.